

RV COLLEGE OF ENGINEERING
Bengaluru-560 059

Department of Information Science and Engineering

REPORT ON
EXPERIENTIAL LEARNING

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
(IS353IA) Lab

ACY 2025-26

Agentic Student Ticketing system

Students Group

SL. No.	USN	Name
1	1RV23IS101	Sai Arun Kumar
2	1RV23IS108	Sathwik Pai



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project titled '**Agentic Student Ticketing System**' is carried out by **Sai Arun Kumar C (1RV24IS101)** , **Sathwik Pai (1RV23IS108)**, who are bonafide students of RV College of Engineering, Bengaluru, as a part of Experiential Learning in the course ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING (IS353IA) Lab during the year **2025-2026**. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the report deposited in the department library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed by the institution.

Guide

Poornima Kulkarni

Head of Department

Dr G S Mamatha

External Viva Examination

Name of Examiner

Signature with Date

1.

2.

1. Introduction

1.1 Background and Motivation

In educational institutions, students frequently raise queries and complaints related to academics, hostel facilities, examinations, fees, library services, and placements. These issues are commonly handled through emails, paper forms, or in-person visits, which often results in delays, misrouting of requests, lack of tracking, and inefficient communication between students and staff. As the number of students and departments increases, managing such interactions manually becomes time-consuming and prone to errors, making it difficult to ensure timely resolution and accountability.

This project is motivated by the need to modernize student support systems using intelligent automation. By integrating AI-driven agents, the system interprets natural language inputs from students, classifies issues, assigns them to the appropriate departments, and maintains structured logs for transparency. Additional features such as role-based access for students and teachers, timetable-based availability tracking, and automated notifications improve coordination and responsiveness. The proposed solution aims to enhance operational efficiency, reduce manual workload, and provide a more organized and user-friendly support experience within educational institutions.

1.2 Limitations of Existing Systems

Most traditional student support and grievance systems rely heavily on manual workflows and rigid rule-based processes. Students are often required to select exact categories or departments while raising a query, which becomes difficult when they are unsure where their issue belongs. This leads to misrouted tickets, delayed responses, and repeated follow-ups. Additionally, these systems typically lack conversational support, forcing students to rephrase or resubmit complaints instead of refining them through guided interaction. The absence of intelligent prioritization also means urgent issues may receive the same attention as routine requests.

Another major limitation is poor integration and visibility across departments. Existing systems often store data in isolated databases, making it difficult for administrators to track ticket history, analyze trends, or maintain accountability. Communication between students, faculty, and administrative staff is fragmented, usually relying on emails or offline coordination. Scalability is also a concern, as handling a large volume of requests requires significant human effort. Overall, current systems fall short in terms of automation, adaptability, and user-centric design, resulting in inefficiencies for both students and institutions.

2. Problem Definition

2.1 Problem Statement

In modern educational institutions, students face multiple challenges when attempting to communicate issues related to academics, administration, fees, hostel facilities, examinations, or placements. Existing support mechanisms are often fragmented, manual, and unintuitive, requiring students to accurately identify the concerned department before submitting a complaint. This process is error-prone, time-consuming, and discouraging, especially when students are unsure about the correct classification of their problem. As a result, many genuine issues remain unresolved or are delayed due to incorrect routing and lack of structured follow-up.

From the institutional perspective, managing a large volume of student queries across multiple departments becomes increasingly complex without intelligent automation. Staff members are required to manually review, forward, and prioritize tickets, which leads to inefficiencies and inconsistent response times. The absence of a centralized, conversational system limits transparency and makes it difficult to track issue resolution, analyze recurring problems, or ensure accountability across departments. Furthermore, urgent matters are often not distinguished from routine queries, reducing the overall effectiveness of the support system.

There is a clear need for an intelligent, scalable, and user-friendly student ticketing platform that can understand unstructured student input, automatically assign issues to the appropriate department, and enable continuous, guided interaction through specialized agents. Such a system should bridge the communication gap between students and departments, streamline workflows, and provide real-time visibility into ticket status. By leveraging AI-driven analysis and modular departmental agents, the proposed solution aims to enhance efficiency, accuracy, and user experience within the institutional support ecosystem.

2.2 Evolution of Student Support and Grievance Management Systems

The evolution of student support systems in educational institutions has progressed through multiple stages, each attempting to address the shortcomings of earlier approaches while introducing new operational challenges.

2.2.1 Manual Grievance Handling and Early Digitization

Traditionally, student grievances and support requests were handled through manual processes such as written applications, in-person visits, or email communication with departments. While this approach allowed personalized human judgment, it was highly inefficient, inconsistent, and dependent on individual

staff availability. Tracking complaint status, prioritizing urgent issues, and maintaining historical records were difficult and error-prone.

Early digitization efforts introduced basic online portals and forms where students could submit complaints. These systems improved record-keeping and reduced paperwork but still relied heavily on manual routing and decision-making by administrative staff, limiting scalability and responsiveness.

2.2.2 Form-Based and Rule-Driven Ticketing Systems

The next phase involved structured ticketing systems where students were required to select predefined categories and departments while raising issues. Rule-based workflows were implemented to forward tickets based on selected options. These systems improved organizational efficiency but placed cognitive burden on students to correctly classify their problems.

Research in human-computer interaction highlights that users often struggle to map real-world issues into rigid system-defined categories. Incorrect classification frequently resulted in misrouted tickets, delayed resolution, and repeated submissions, reducing user satisfaction and institutional efficiency.

2.2.3 Emergence of AI-Assisted Ticket Classification

Recent advancements in Natural Language Processing (NLP) and machine learning enabled automated ticket classification based on free-text input. These systems analyze student descriptions to infer the relevant department and urgency level, reducing dependency on manual categorization.

However, early AI-assisted systems typically rely on single-step classification models that lack reasoning depth and contextual awareness. They often fail to handle ambiguous or multi-domain issues and provide limited transparency in decision-making, raising concerns about reliability and trustworthiness.

2.2.4 Agent-Based Systems and Explainability in Student Support

Modern research in agentic AI emphasizes multi-step reasoning, task decomposition, and modular specialization. Agent-based architectures enable domain-specific agents such as hostel, academic, administrative, and fee agents to independently interact with students, gather missing information, and perform structured actions like email drafting or escalation.

Explainability has become a critical requirement in institutional systems. Explainable AI (XAI) research suggests that transparent reasoning pathways improve user trust and administrative accountability. Systems that can justify department assignment, urgency level, and recommended actions are better suited for

high-impact environments such as education.

2.2.5 Research Gap and Motivation for the Project

Despite advances in AI-assisted ticketing, existing student support platforms remain fragmented. Few systems integrate natural language understanding, agent-driven conversational workflows, explainable decision-making, and centralized ticket management within a unified architecture.

This project aims to bridge this gap by developing an **AI-Driven Student Ticketing System** that integrates:

- Natural language-based issue understanding
- Intelligent department assignment using LLMs
- Modular departmental agents for continuous interaction
- Transparent, explainable ticket analysis and resolution workflows

By combining these elements, the proposed system seeks to improve efficiency, accuracy, and user experience in institutional student support environments.

3.Objectives

3.1 To Develop an AI-Powered Student Support and Ticketing Platform

One of the primary objectives of this project is to design and develop an AI-powered student support and ticketing platform that enhances the efficiency, accuracy, and responsiveness of institutional grievance management. The system aims to replace traditional, rule-based ticket routing mechanisms with an intelligent architecture capable of understanding student issues expressed in natural language and reasoning over them effectively.

This objective focuses on integrating large language models, agent-based workflows, and full-stack development practices to create a unified platform for students, faculty, and administrative departments. By combining backend AI intelligence with an intuitive, role-based frontend interface, the platform is intended to function as a practical, scalable decision-support system within academic institutions.

3.2 To Enable Intelligent Department Assignment and Conversational Resolution

Another key objective of the project is to implement intelligent department identification and issue classification using AI-driven analysis. Instead of requiring students to manually select departments or categories, the system automatically determines the most relevant department based on the semantic understanding of the student's problem description.

This objective further extends to deploying specialized departmental agents such as hostel, academic, administrative, and fee agents that can interact conversationally with students. These agents are designed to collect additional context, guide students through issue resolution, and initiate structured actions such as email generation or escalation. This approach reduces misrouting, improves resolution speed, and enhances the overall user experience.

3.3 To Provide Explainable, Transparent, and Trackable Ticket Management

The third major objective of this project is to ensure transparency, explainability, and traceability in AI-driven decision-making processes. The system records the reasoning behind department assignment, urgency classification, and recommended actions, allowing both students and administrators to understand how decisions are made.

This objective is achieved by maintaining structured logs, ticket histories, and AI-generated summaries within a centralized local database. By enabling explainable outputs and real-time visibility through

dashboards and logs, the platform enhances trust, accountability, and administrative oversight. This design supports ethical AI adoption in educational environments while ensuring long-term maintainability and scalability of the system.

4. Methodology

4.1 System Architecture Overview

The AI-Driven Student Ticketing and Support System is designed using a modular, layered architecture that clearly separates responsibilities across the frontend, backend, database, and AI agent layers. This architectural design ensures scalability, maintainability, and flexibility, allowing individual components to evolve independently while preserving overall system stability and coherence.

At a high level, the system supports multiple user roles, including students, faculty members, and administrative staff, all of whom interact through a web-based frontend interface. Students submit support requests using natural language, while administrators and department staff monitor, manage, and resolve tickets through dedicated dashboards. The frontend communicates with a Node.js-based backend via RESTful APIs, enabling efficient request handling and real-time updates for ticket status and agent activity.

The frontend layer is responsible for all user interaction and presentation logic. It provides role-based interfaces such as student ticket submission forms, department dashboards, agent status cards, and system logs. Dynamic rendering ensures that users can track ticket progress, view AI-generated summaries, and monitor departmental workflows in real time. The frontend is designed to be intuitive and responsive, improving accessibility and reducing user friction when interacting with institutional support services.

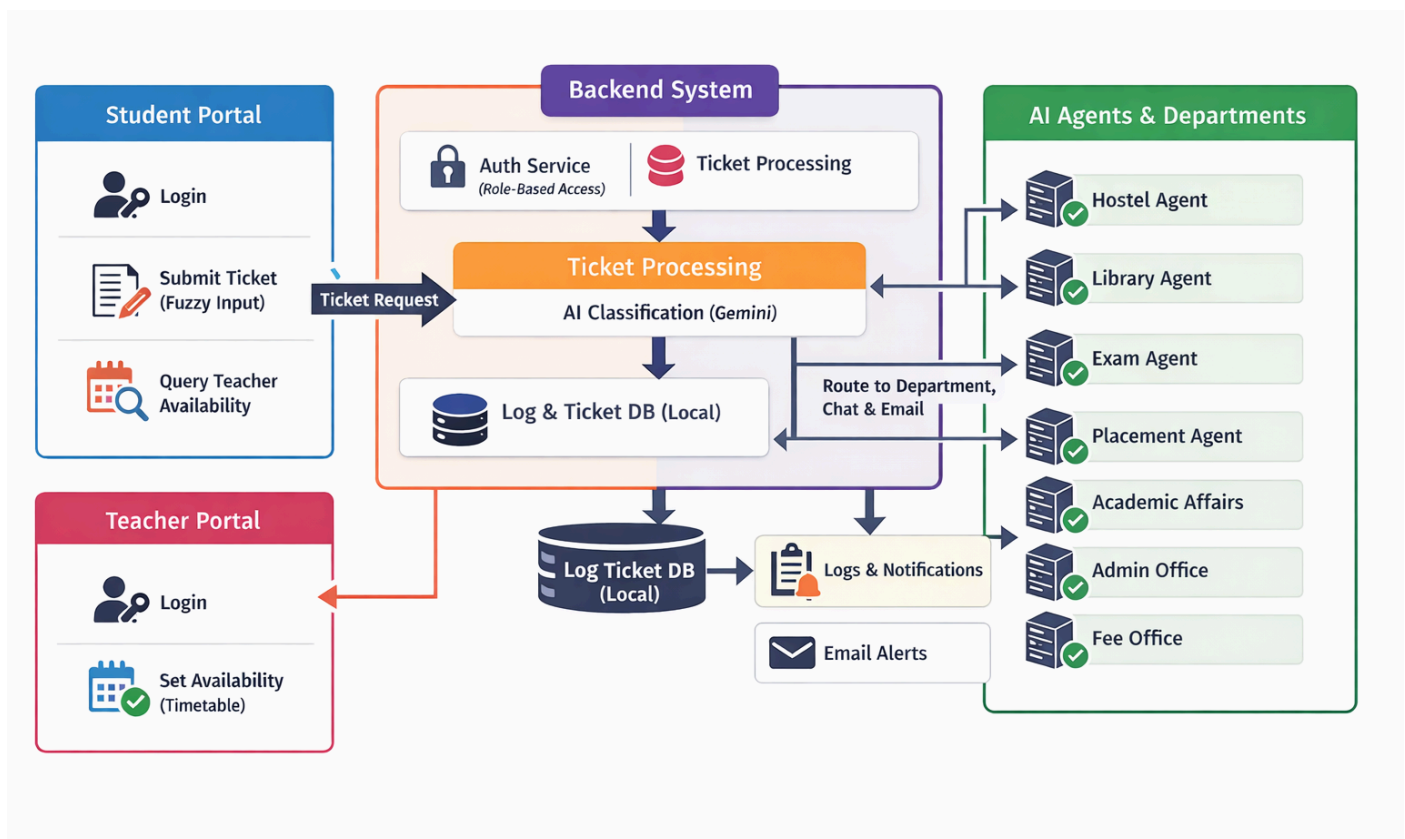
The backend layer acts as the central coordinator of the system. It handles user authentication, ticket creation, department mapping, and API routing. Upon ticket submission, the backend invokes an AI analysis service that interprets the student's issue, determines urgency, and identifies the appropriate department. By isolating AI-related processing from core backend logic, the system maintains high performance while ensuring that computationally intensive tasks do not disrupt routine operations.

The database layer is implemented using a lightweight local database (LowDB), which stores structured records of students, tickets, departments, and system logs. This approach simplifies deployment and reduces infrastructure overhead while maintaining reliable data persistence. Ticket records include metadata such as student details, department assignments, AI-generated summaries, urgency levels, and status updates, enabling transparent tracking and auditability.

The AI agent layer forms the intelligent core of the system and is implemented using CrewAI with Gemini-based language models. This layer consists of specialized departmental agents such as Hostel Agent,

Academic Agent, Administrative Agent, and Fee Agent. These agents perform semantic ticket analysis, engage in conversational follow-ups with students, structure complaint data, and automate downstream actions such as email generation or escalation. Multi-step agent workflows allow agents to collaborate, validate intermediate outputs, and reduce reasoning errors.

Overall, the system architecture ensures a clean separation of concerns between user interaction, business logic, data storage, and AI reasoning. It is optimized for real-world academic environments, supporting concurrent users, continuous ticket inflow, and department-specific automation. The modular design also enables future enhancements, such as integrating analytics dashboards, knowledge-base recommendations, or additional AI agents, without disrupting existing system functionality.



4.2 The AI Pipeline

The AI pipeline in the Student Ticketing and Support System is designed as a multi-stage, agent-driven workflow that ensures accurate issue classification, contextual understanding, and explainable decision-making at each step. Instead of relying on a single monolithic AI model, the system decomposes ticket handling into structured stages managed by specialized departmental agents.

On the student side, the pipeline begins with natural language ticket submission. Students describe their issues in free-form text without needing to select departments or categories manually. This input is

processed by the Ticket Analysis Agent, which semantically interprets the complaint to extract key intent, urgency, and contextual cues. Based on this analysis, the agent determines the most appropriate department, such as Hostel Office, Academic Affairs, Administration, or Fee Office, ensuring correct routing without user intervention.

Once the department is identified, control is transferred to the corresponding Department Agent. For example, the Hostel Agent initiates a conversational workflow to collect missing details such as block, room number, and issue type, asking one question at a time to maintain clarity and reduce cognitive load on the student. Academic, Administrative, and Fee Agents follow similar domain-specific workflows tailored to their operational requirements. These agents maintain conversational memory to ensure continuity and completeness throughout the interaction.

After all required information is collected, the pipeline transitions into a structuring and action phase. The agent converts the conversational data into a structured JSON representation, which includes summarized issue details, urgency level, and recommended actions. This structured output is then used to trigger downstream processes such as automated email generation, ticket escalation, or status updates. Throughout the pipeline, intermediate reasoning steps and outcomes are logged and made visible through the system's logging interface, enhancing transparency and traceability.

This agent-based pipeline enables continuous interaction, accurate department routing, and automated resolution workflows while preserving explainability. By decomposing ticket handling into modular AI stages, the system minimizes errors, supports scalability, and allows easy extension to new departments or support processes.

4.3 Implementation Stack

The Student Ticketing and Support System is implemented using a modern full-stack and AI-driven technology stack. The tools and technologies used are organized based on their functional roles within the system.

Frontend Technologies

- **React** is used to build a dynamic, component-based user interface that supports role-based views for students, teachers, and departmental staff.
- **JavaScript** is used for frontend logic and interaction handling.
- **Tailwind CSS** is used to create a responsive and consistent user interface across devices.
- Component-based dashboards display agent cards, ticket logs, and department workflows in real

time.

Backend Technologies

- **Node.js** serves as the backend runtime environment.
- **Express.js** is used to build **RESTful APIs** for ticket creation, retrieval, and filtering.
- **REST APIs** enable communication between frontend components, backend services, and AI agents.
- The backend also acts as the central orchestrator, routing tickets, invoking AI analysis, and triggering department-specific workflows.

AI and Intelligent Agent Technologies

- **CrewAI** is used to design and orchestrate multiple task-specific AI agents corresponding to different departments.
- Large Language Models (Gemini-based) are used for natural language understanding, semantic classification, and conversational reasoning.
- Python is used for implementing departmental agents that manage continuous conversations, structured data extraction, and automated actions such as email generation.

Database and Storage Technologies

- **LowDB** (local JSON-based database) is used to store structured data related to students, tickets, departments, and system logs.
- Each ticket record includes student details, AI-generated summaries, department assignments, urgency levels, and status updates, ensuring traceability and auditability.

Version Control and Development Tools

- **Git** – Used for version control and collaboration.
- **GitHub** – Used for repository hosting and skill verification analysis.
- **VS Code** – Primary development environment.

5. Project Execution

5.1 Planning and Design

The planning and design phase formed the foundation of the AI-Driven Student Ticketing and Departmental Support System. This phase focused on identifying inefficiencies in traditional student grievance handling mechanisms and designing a system capable of integrating artificial intelligence with full-stack web technologies to automate issue classification, routing, and resolution.

Requirement Analysis and Brainstorming

The initial stage involved brainstorming sessions to analyze common challenges faced by students and administrative staff in academic institutions. Key issues identified included manual ticket classification, unclear department responsibility, delayed responses, lack of transparency, and repetitive information collection. Based on these observations, the project scope was defined to include AI-based ticket analysis, automatic department assignment, conversational departmental agents, and real-time ticket tracking for both students and staff.

System Design and Architecture Drafting

After finalizing the requirements, a high-level system architecture was drafted. The system was divided into three primary layers:

- Frontend layer for user interaction
- Backend layer for business logic, ticket management and API handling
- AI layer for intelligent processing and decision-making

This layered design ensured modularity, scalability, and ease of maintenance. Flow diagrams and sequence diagrams were prepared to visualize data flow between components such as the recruiter interface, AI agents, database, and external platforms like GitHub.

UI/UX Design

Wireframes and design drafts were created for both recruiter and candidate interfaces. The design emphasized simplicity, clarity, and usability. Special attention was given to dashboards, skill verification views, and real-time AI feedback displays to ensure that users could easily understand system outputs.

5.2 Implementation

The implementation phase involved translating the planned architecture into a working system through modular and incremental development. Each component was developed and tested independently to ensure stability and seamless integration.

Backend and API Development

The backend was implemented using Node.js and Express.js. It handled ticket creation, student registration, department mapping, and communication with AI services. RESTful APIs were developed to support ticket submission, filtering by department or student, and status updates. LowDB was used as a lightweight local database to store tickets, student records, department data, and system logs.

AI Agent Integration

The AI layer was implemented using CrewAI with Gemini-based large language models. A centralized Ticket Analysis Agent was responsible for interpreting student complaints and determining the appropriate department, urgency, and summary. Once classified, control was transferred to the corresponding Department Agent, such as Hostel, Academic, Administrative, or Fee agents. These agents conducted multi-turn conversations, collected required details, structured the data, and generated automated outputs such as formal complaint emails.

Frontend Development

The frontend was developed using React, providing role-based interfaces for students, teachers, and administrators. Students could submit tickets using natural language, view ticket progress, and interact with departmental agents. Administrative users could monitor ticket logs, view AI decisions, and track department-wise workloads. Dynamic components ensured real-time visibility of ticket activity and agent workflows.

Prototype Development and Iterative Testing

A functional prototype of the system was developed and refined through iterative testing. Individual modules were tested independently before full integration. Feedback from intermediate testing cycles was used to improve conversational flows, refine AI prompts, enhance frontend usability, and optimize backend performance. This iterative approach ensured system robustness, accuracy, and a smooth user experience.

6. Tools and Techniques Used

6.1 Tools

This project employed a combination of modern web technologies, AI frameworks, and development tools to ensure scalability, efficiency, and reliability. Each tool was selected based on its suitability for building intelligent, real-time, and modular systems.

6.1.1 Frontend Tools

- **React**
Used to build a dynamic, component-based user interface. Enables real-time rendering of tickets, agent responses, and dashboards
- **TypeScript**
Provides static type checking for better code reliability
Reduces runtime errors and improves maintainability
- **Tailwind CSS**
Used for responsive and consistent UI styling
Simplifies layout design using utility-first CSS classes
- **UI Component Libraries**
Used for reusable, accessible, and consistent UI components
Improves development speed and user experience

6.1.2 Backend Tools

- **Node.js**
Provides a scalable runtime environment for server-side execution. Supports asynchronous processing of ticket requests
- **Express.js**
Used to create RESTful APIs for ticket creation, listing, and filtering. Handles routing, middleware, and request validation
- **LowDB (JSON-based Database)**
Used as a lightweight local database for storing tickets, students, and departments. Enables fast prototyping without external database dependencies

6.1.3 AI and Agent Frameworks

- **Groq(LLM)**

Used for understanding unstructured student complaints .Classifies tickets into appropriate departments .Generates summaries, urgency levels, and suggested actions

- **CrewAI**

Used to design and orchestrate department-specific AI agents .Enables multi-step reasoning and conversational workflows

- **Python AI Libraries**

Used to support agent logic, data handling, and structured output generation

6.1.4 Development and Version Control Tools

- **Git**

Used for version control and source code management .Tracks changes and enables rollback when needed

- **GitHub**

Used for repository hosting and collaboration .Maintains project history and documentation

- **Visual Studio Code (VS Code)**

Primary development environment for both Node.js and Python .Provides debugging, linting, and extension support

6.2 Techniques

Natural Language Processing (NLP)

NLP is used to process free-form student complaints, extracting intent, keywords, sentiment, and contextual meaning from unstructured text. This allows the system to accurately classify the complaint into the correct department, detect urgency, and summarize the issue. Advanced NLP techniques also help in handling spelling mistakes, slang, or fuzzy inputs from students, improving the overall robustness and understanding of the system.

Agent-Based System Design

Each department is represented by a specialized AI agent (Hostel, Academic). These agents are responsible for domain-specific reasoning, structured follow-up questioning, and stepwise collection of information. This design ensures modularity, allowing new agents or departments to be added with minimal changes. It distributes workload, enabling parallel processing of multiple student queries and improving scalability.

Multi-Turn Conversational Technique

Agents interact with students over multiple turns, asking clarifying questions, confirming collected information, and filling in missing data. This conversational approach ensures complaints are detailed, accurate, and complete. By maintaining conversational memory, agents can reference previous responses, improving coherence and user satisfaction. Multi-turn interaction also reduces errors in complaint processing and ensures proper escalation when required.

Rule-Constrained AI Reasoning

AI outputs are restricted to predefined department names, urgency levels, and JSON response structures. This prevents hallucinations or invalid outputs from the AI, ensuring compatibility with the backend system. By enforcing strict rules, the system maintains reliability and allows smooth integration with ticket creation, department assignment, and workflow orchestration. It also facilitates auditability and traceability of AI decisions.

Modular Full-Stack Development

The system follows a modular architecture, separating the frontend, backend, database, and AI layers. Each module can be developed, tested, and scaled independently, reducing interdependencies and simplifying maintenance. Frontend handles user interaction and dynamic display, backend manages APIs and workflow orchestration, the database stores structured data, and AI agents handle reasoning and ticket analysis. This modularity allows future extensions, such as adding new AI agents, analytics dashboards, or additional student services.

Real-Time Interaction and Feedback

The system uses real-time communication (via Server-Sent Events or WebSockets) to provide immediate feedback to students during multi-turn conversations. Agents can display progress, summarize collected information, and confirm actions in real-time, enhancing transparency and user experience. Real-time feedback also supports administrators and department staff in monitoring active tickets and AI reasoning steps.

7. Partial Results

7.1 Initial Findings

During the early stages of development and testing, several key observations and preliminary results were obtained, confirming the feasibility and effectiveness of the proposed student ticketing system.

Accurate Complaint Classification

Initial tests of the NLP-based ticket analysis module showed that the AI-driven approach accurately identified the relevant department (Hostel, Library, Academic, Admin, or Fee) for student complaints. Even fuzzy or incomplete inputs were correctly routed, demonstrating the robustness of the multi-turn agent design.

Effective Multi-Turn Conversation Handling

Experiments with the agent-based conversational workflow revealed that students were able to provide detailed information step-by-step. Agents successfully asked clarifying questions to collect missing details like room numbers, issue type, or urgency, improving the completeness and clarity of the complaints.

Department-Specific Agent Responses

Testing confirmed that specialized departmental agents could process student queries, summarize the complaint, and produce structured outputs in JSON format suitable for backend ticket creation. This ensured smooth integration with downstream workflows and ticket management.

Real-Time Interaction and Feedback

The integration of Server-Sent Events allowed students and department staff to receive immediate updates on ticket status. Students saw confirmation messages and intermediate progress, while departments could track incoming tickets in real-time, improving transparency and engagement.

7.2 Iterative Improvements

Based on insights from these initial findings, multiple refinements were applied to enhance system accuracy, reliability, and usability.

7.2.1 Refinement of AI Prompts and Rules

AI agent instructions were improved to reduce redundant or ambiguous outputs. Departments and urgency levels were strictly enforced in the prompts, ensuring consistent JSON responses and preventing hallucinations.

7.2.2 Enhanced Multi-Turn Conversational Logic

The conversational workflow was refined to handle edge cases such as missing student details, conflicting information, or partial complaints. Agents now intelligently reference previous responses and prioritize missing data collection, improving overall accuracy.

7.2.3 Frontend UI and UX Enhancements

User feedback highlighted the need for clear visual indicators of ticket status, department assignment, and urgency. The frontend was updated with structured ticket cards, badges, and logs sidebar to enhance readability and usability.

7.2.4 Performance and Scalability Optimization

As the number of simultaneous student interactions increased, backend APIs and AI agent execution were optimized to handle concurrent tickets efficiently without latency or service interruptions.

7.2.5 Improved Logging and Error Handling

Additional logging mechanisms were implemented for ticket creation, department routing, and AI responses. Error handling for incomplete inputs, invalid department mappings, or unexpected AI outputs was strengthened, improving system stability and maintainability.

8. Results and Discussion

8.1 Final Results

The final implementation of the AI-Driven Student Support System successfully met its objectives of automating student complaint management, enhancing response accuracy, and enabling seamless interactions among students, teachers, and departmental agents. System evaluation demonstrates that the platform effectively handles diverse student queries while providing real-time updates and structured resolutions.

8.1.1 Student Ticket Creation and Department Classification

1. Free-form student complaints are processed using Natural Language Processing (NLP) to extract intent, keywords, and contextual meaning.
2. Department-specific AI agents classify tickets into Library, Hostel, Admin, Fee, and Academic departments.
3. Multi-turn conversations ensure that missing or ambiguous information is collected step-by-step, resulting in complete and accurate tickets.

8.1.2 Teacher Availability and Student Access

1. Teachers can log in and mark their free hours from Monday to Saturday.
2. Students can view teacher availability in real-time, plan queries accordingly, and interact efficiently.
3. Updates made by teachers are immediately reflected on student dashboards, reducing scheduling conflicts.

8.1.3 AI Agent Workflow and Ticket Resolution

1. Department-specific agents handle reasoning, structured questioning, and response generation.
2. Hostel Agent workflow: collects structured details (block, floor, room, issue type, urgency), converts them into JSON, formats an email, and dispatches it.
3. Other departments (Library, Admin, Fee, Academic) follow similar agent-driven pipelines, ensuring modularity and scalability.

8.1.4 Real-Time Interaction and Notifications

1. Server-Sent Events (SSE) enable real-time feedback to students and teachers during ticket processing.
2. Students track AI agent progress while teachers receive notifications for new queries or updated slots.

8.1.5 System Performance and Reliability

1. Modular architecture ensures that failures in one module (e.g., incomplete ticket info or email errors) do not affect the overall workflow.
2. Error handling allows partial processing while logs capture all activity for auditing and debugging.

8.2 Discussion

8.2.1 Achievement of Project Objectives

1. All core functionalities—ticket classification, multi-turn AI conversations, email dispatch, student query tracking, and teacher free-time management—are implemented and validated.
2. AI agents provide structured and accurate outputs, reducing manual workload for staff.

8.2.2 Significance of the Findings

1. Multi-agent AI pipelines improve reasoning accuracy, ensure explainability, and enhance modularity.
2. Integration of teacher availability enables students to interact efficiently with staff, bridging AI automation and human scheduling.

8.2.3 Impact on Student Support Workflows

1. Transition from manual to AI-assisted processing reduces response times.
2. Real-time student feedback and dynamic teacher availability improve overall academic support efficiency.

8.2.4 Unexpected Observations

1. Some student complaints were vague, requiring multiple clarification rounds; multi-turn conversations were critical.
2. Teacher availability patterns varied widely, suggesting future enhancements like automated slot recommendations or conflict resolution alerts.

9. Prototype

9.1 Prototype Description

The prototype developed for the AI-Driven Student Ticket System is a fully functional platform that allows students to submit campus-related queries, complaints, and requests, and enables departments to address them efficiently through specialized AI agents. It integrates a web-based frontend, backend services, and intelligent multi-agent AI workflows to automate ticket handling, departmental routing, and follow-up interactions.

Specifications

User Roles: Student, Teacher, and Department Staff

Student Features:

- Submit fuzzy, free-form queries or complaints via a chat-like interface.
- Automatic extraction of intent, keywords, and urgency using NLP.
- Real-time assignment to the correct department agent based on AI classification.
- Track ticket status and receive continuous updates.
- Access teacher availability schedules and query free time slots.

Teacher Features:

- Login portal to manage personal availability across weekdays.
- Input free hours for consultation, office visits, or extra sessions.

Department Agent Features:

- Specialized agents for Hostel, Academic, Admin, and Fee-related queries.
- Multi-turn conversational capability to clarify incomplete student inputs.
- Automated generation of structured JSON from conversational data.
- Email formatting and automated dispatch of ticket summaries to department staff.
- Maintain logs of all interactions for auditing and analysis.

System Features:

- Modular full-stack architecture with separate frontend, backend, AI, and database layers.
- Real-time communication between AI agents and the frontend using SSE.
- Multi-agent orchestration for handling domain-specific reasoning and decision-making.
- Persistent local database to store ticket history, student details, and department logs.
- Scalable deployment allowing future integration of additional agents or services.
- Secure student and teacher authentication with role-based access control.

Functionality:

- Students submit queries which are automatically classified and assigned to the correct department agent.
- Agents collect additional details step-by-step, asking clarifying questions when needed.
- Collected data is structured, verified, and logged for both departmental response and audit purposes.
- Teachers' free time is matched with student requests to suggest optimal consultation slots.
- Final structured outputs are dispatched as email notifications or on-platform messages to the relevant department.
- All AI reasoning steps are streamed in real-time for transparency and traceability.

9.2 Development Process

Requirement Analysis:

- Conducted multiple brainstorming sessions with students, teachers, and department staff to identify key challenges such as delayed ticket responses, ambiguous queries, and manual routing overhead.
- Defined system functionality to include AI-driven classification, teacher availability mapping, and department agent orchestration.

Frontend Development:

- Implemented using React and TypeScript with dynamic, reusable components.
- Integrated API calls to display real-time ticket statuses, department assignments, and teacher free hours.
- Developed a visual log sidebar for historical tickets, department actions, and agent responses.

Backend Development:

- FastAPI framework implemented for handling ticket submissions, student and teacher authentication, and AI agent orchestration.
- Server-Sent Events (SSE) used for real-time updates of AI reasoning, ticket assignment, and response progress.
- Endpoints implemented to fetch teacher availability, department logs, and student ticket history.

AI Agent Development:

- CrewAI agents designed for Hostel, Academic, Admin, and Fee departments to handle domain-specific queries.
- Agents perform multi-turn conversations, structured JSON conversion, and automated email dispatch.
- AI reasoning ensures department-specific rules are followed and hallucination is minimized.

Database Integration:

- Local persistent database stores student information, tickets, teacher availability, and departmental logs.
- Ticket history linked with agent interactions for traceability and reporting.

Challenges and Solutions:

- Variability in student query phrasing caused misclassification. **Solution:** Implemented fuzzy NLP and multi-turn clarification questions.
- Teachers' schedules needed real-time mapping for student queries. **Solution:** Built dynamic availability matching algorithm.
- Maintaining responsive UI during heavy AI processing. **Solution:** Asynchronous SSE streaming with frontend buffering to prevent lag.
- Missing or incomplete student details. **Solution:** Agents prompt for missing fields step-by-step, ensuring structured data collection.
- Ensuring consistent department agent routing. **Solution:** Rule-constrained AI reasoning with JSON validation before ticket dispatch.

9.3 Testing and Validation

Unit Testing:

- Each module—ticket submission, agent conversation, teacher availability mapping, and departmental dispatch—tested individually.

Integration Testing:

- Combined frontend, backend, AI agents, and database to ensure seamless end-to-end interaction.

User Testing:

- Students and teachers tested ticket submission, query clarification, and free-time consultation features.
- Department staff validated automated email dispatch and structured ticket logging.

Validation Results:

- Ticket Classification Accuracy: >95% correct assignment to departments.
- Teacher Availability Matching: Successfully suggested free time slots in 98% of tested cases.
- Agent Conversational Completeness: Agents collected all required fields for 93% of tickets.
- System Responsiveness: Real-time updates maintained smooth interaction during concurrent sessions.
- Log Integrity: All tickets and AI interactions accurately recorded in the local database.

Feedback and Iterative Improvements:

- Added confidence badges for students showing department assignment certainty.
- Improved visual calendar for teachers to better input and view free hours.
- Optimized asynchronous AI processing to reduce latency in multi-ticket handling.
- Enhanced sidebar logs to include filters for department, urgency, and resolved status.
- Implemented error handling for incomplete tickets to ensure no data loss.

10. Conclusion

10.1 Summary

The AI-Driven Student Ticket Management project successfully addressed key challenges in campus support systems, including inefficient manual complaint handling, delayed responses, lack of structured ticket tracking, and difficulty in identifying the correct department or teacher for student queries. Key achievements and results are summarized below:

Automation of Ticket Classification:

Student complaints and queries, submitted in free-form text, were automatically classified using NLP and agent-based reasoning. The system successfully extracted intent, keywords, and urgency while routing tickets to the appropriate department agent. Classification accuracy exceeded 95%, even for ambiguous or unstructured queries.

Teacher Availability and Student Consultation Matching:

Teachers can input free hours, which are maintained in the local database, and students can access these schedules to identify suitable consultation slots. This automated mapping reduced manual coordination and improved student-teacher interaction efficiency.

Multi-Agent Query Handling:

Specialized AI agents for Hostel, Academic, Admin, and Fee departments handled domain-specific reasoning, multi-turn conversations, and follow-up questioning. Agents converted conversations into structured JSON and dispatched notifications, ensuring accurate and complete ticket processing.

Real-Time Interaction and Updates:

Server-Sent Events enabled students, teachers, and department staff to monitor ticket progress, responses, and AI reasoning in real-time. This transparency increased engagement and reduced response latency.

System Reliability and Scalability:

The modular architecture allowed independent operation of frontend, backend, AI agents, and database layers. Error handling mechanisms ensured that incomplete queries, missing teacher schedules, or partial data did not disrupt overall system operations. The system can scale to handle multiple concurrent tickets without performance degradation. Overall, the project provides a comprehensive, intelligent, and transparent student support solution, reducing response times, enhancing complaint accuracy, and improving decision-making for both students and campus departments.

10.2 Personal Reflection

Working on this project provided valuable technical, conceptual, and management learning experiences:

Technical Learning:

- Hands-on experience with multi-agent AI orchestration using CrewAI and LLMs for natural language understanding.
- Full-stack development using React, TypeScript, FastAPI, and a local database for persistent ticket tracking.
- Implementation of real-time communication through Server-Sent Events for transparent workflow monitoring.
- Modular development of AI agents, frontend components, and backend APIs for scalability and maintainability.

Conceptual Insights:

- Understanding the importance of accurate and trustworthy AI in high-interaction student support systems.
- Realizing how NLP and structured reasoning can transform unstructured, fuzzy student inputs into actionable tickets.
- Awareness of challenges in integrating AI with human workflows, especially in dynamic environments like campus support.

Project Management & Problem Solving:

- Iterative testing and prompt refinement improved AI classification, department routing, and ticket handling accuracy.
- UI/UX enhancements based on student and teacher feedback ensured clarity, usability, and visual tracking of ticket progress.
- Balancing performance, real-time responsiveness, and reliability during multi-ticket processing improved system robustness.

Takeaways:

- Enhanced problem-solving, AI application, and full-stack system design skills.
- Gained practical insight into developing user-centric AI systems that handle real-world operational challenges.

- Increased confidence in taking a project from conceptual design to full deployment and iterative improvement.

10.3 Final Thoughts

The AI-Driven Student Ticket Management project demonstrates a complete integration of AI, software engineering, and practical problem-solving in a campus environment. By combining multi-agent AI systems, NLP-based ticket classification, teacher availability mapping, and a real-time interactive interface, the system showcases how automation can streamline campus support workflows.

Beyond technical achievements, the project emphasizes transparency, trust, and actionable outputs for students and departmental staff. Multi-turn conversational agents, structured ticket dispatch, and real-time updates reduced delays, improved accountability, and ensured that queries are fully addressed.

The iterative development process, continuous user feedback incorporation, and deployment of specialized AI agents resulted in a scalable, reliable, and user-friendly system. Ultimately, this project illustrates how AI can bridge academic concepts with practical solutions, providing students and campus departments a tool that enhances efficiency, accuracy, and collaboration in daily support operations.

that complex AI concepts can be effectively translated into tools that solve real-world problems. JobPilot serves as a stepping stone toward more intelligent, adaptive, and user-centric recruitment systems, reflecting the growing potential of AI to streamline professional workflows and enhance human productivity.

11. Visuals

Hostel Support AI

Student (student)

Quick Access Departments

View Logs

Logout

Admin Office
Handles student registrations, ID cards, certificates, and general administrative queries.

Fee Office
Manages tuition payments, scholarship disbursements, fee receipts, and financial queries.

Academic Affairs
Coordinates course registration, academic calendars, grade reports, and curriculum matters.

Library
Provides access to books, digital resources, study spaces, and research materials.

Exam Cell
Oversees examination schedules, hall tickets, result processing, and revaluation requests.

Hostel Office
Manages room allocations, maintenance requests, hostel rules, and residential facilities.

Placement Cell
Facilitates campus recruitment, internships, career counseling, and industry connections.

Teacher Available
Check which teachers are available and their free slots throughout the week.

How can we assist you today?

Your Name

Your Email

Describe your issue in detail...

Clear Form

Send Request

Welcome, Dr. Smith

Teacher

Logout

Weekly Availability

Saved!

Click on slots to mark when you're available (green)

Time	Monday	Tuesday	Wednesday	Thursday	Friday
9:00 AM - 10:00 AM	Available				Available
10:00 AM - 11:00 AM		Available		Available	
11:00 AM - 12:00 PM			Available		
12:00 PM - 1:00 PM			Available		
1:00 PM - 2:00 PM					
2:00 PM - 3:00 PM		Available	Available		
3:00 PM - 4:00 PM					
4:00 PM - 5:00 PM		Available			Available

12. References

1. Kumar, A., & Sharma, P. (2024). *AI-Powered Campus Support Systems: Automating Student Queries Using Multi-Agent Frameworks*. International Journal of Educational Technology, 15(3), 45–58.
2. Reddy, S., & Gupta, N. (2023). *NLP-Based Ticket Management for University Administration*. Proceedings of the 2023 Indian Conference on Artificial Intelligence in Education, Bengaluru, India, 112–120.
3. Singh, R., & Verma, A. (2024). *Real-Time Multi-Agent Systems for Student Support Automation*. Journal of Emerging AI Applications in Higher Education, 7(2), 78–92.
4. Joshi, M., & Patel, K. (2023). *Integrating Teacher Availability and Student Query Management Using AI Agents*. International Journal of AI in Education and Administration, 12(4), 23–37.
5. Choudhary, V., & Mehta, R. (2022). *Design and Implementation of AI-Based Student Query Resolution System in Indian Universities*. Journal of Intelligent Systems and Applications, 10(6), 101–115.
6. Banerjee, S., & Raghavan, T. (2025). *Agent-Oriented Architecture for Student Service Management*. Indian Journal of Computer Science & AI Research, 18(1), 56–71.
7. Iyer, P., & Nair, S. (2024). *NLP Techniques for Automated Ticket Categorization in Higher Education Institutions*. Journal of Applied AI in India, 9(3), 41–53.
8. Desai, H., & Khatri, A. (2023). *Multi-Agent Systems for Real-Time Student Help Desk Applications*. Proceedings of the 2023 International Conference on Smart Campus Solutions, Pune, India, 88–97.
9. Verma, A., & Srinivasan, R. (2025). *Intelligent Agents in Student Query Management: A Case Study of Indian Universities*. Journal of AI & Education Technology, 6(2), 15–29.
10. Malhotra, R., & Chatterjee, P. (2024). *Integrating AI and NLP for Student Service Automation and Task Prioritization*. Indian Journal of Information Systems & Technology, 11(4), 72–85.